The mathematics of the implemented shifted short recurrence

One algorithm, two implementations, and a crucial residual distinction

This note describes the actual updates in `src/linsys.f90` and `petsc_shifted_pminres.c`. The C routine is a custom PETSc port, not PETSc's built-in `KSPMINRES`. The derivation below assumes exact arithmetic and no breakdown; the last page explains the practical stopping checks.

1. The structure that makes a short recurrence possible

We solve, with no preconditioner and initial guess $x_0 = 0$,

$$Ax = b, \qquad A = H + i\gamma I, \qquad H = H^\dagger, \quad \gamma \in \mathbb{R}. \tag{1}$$

A real energy shift can be absorbed into H. For $\gamma \neq 0$, A is generally *not Hermitian*, but it is normal: $A^\dagger A = AA^\dagger = H^2 + \gamma^2 I$. Also $\mathcal{K}_k(A, b) = \mathcal{K}_k(H, b)$, since adding a scalar multiple of I does not change the Krylov subspace. Thus the basis can be generated by Hermitian Lanczos even though the projected diagonal is complex.

Set $\beta_1 = \|b\|_2$, $v_1 = b/\beta_1$, and $v_0 = 0$ (handle $b = 0$ separately). At iteration j, compute

$$\begin{aligned} q_j &= Av_j, & \alpha_j &= v_j^\dagger q_j, \\ t_j &= q_j - \alpha_j v_j - \beta_j v_{j-1}, & \beta_{j+1} &= \|t_j\|_2, \qquad v_{j+1} = t_j/\beta_{j+1}. \end{aligned} \tag{2}$$

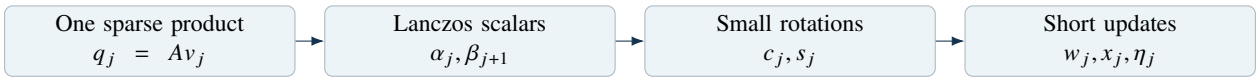
Here $\alpha_j = v_j^\dagger H v_j + i\gamma$. Consequently the imaginary shift cancels in $q_j - \alpha_j v_j$, leaving exactly the Hermitian Lanczos basis recurrence. For $j = 1$, the arbitrary-looking use of β_1 as a coupling is harmless because $v_0 = 0$. In exact arithmetic the v_j are orthonormal.

2. A large sparse problem becomes a small tridiagonal one

Let $V_k = [v_1, \ldots, v_k]$. The recurrence gives

$$AV_k = V_{k+1}\underline{T}_k, \qquad \underline{T}_k = \begin{bmatrix} \alpha_1 & \beta_2 & & \\ \beta_2 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_k \\ & & \beta_k & \alpha_k \\ & & & \beta_{k+1} \end{bmatrix}_{(k+1)\times k}. \tag{3}$$

For $x_k = V_k y_k$, $b - Ax_k = V_{k+1}(\beta_1 e_1 - \underline{T}_k y_k)$. The next page shows how the code updates x_k without storing V_k or explicitly forming this small matrix.



3. The scalar and solution recurrences, exactly as coded

Use $c_0 = c_{-1} = 1$, $s_0 = s_{-1} = 0$, $w_0 = w_{-1} = 0$, and $\eta_0 = \widehat{r}_0 = \beta_1$. After obtaining α_j and β_{j+1}, the code applies

$$\begin{aligned} \delta_j &= c_{j-1}\alpha_j - c_{j-2}s_{j-1}\beta_j, \\ d_j &= \sqrt{\delta_j^2 + \beta_{j+1}^2}, \\ u_j &= s_{j-1}\alpha_j + c_{j-2}c_{j-1}\beta_j, & z_j &= s_{j-2}\beta_j, \\ c_j &= \delta_j/d_j, & s_j &= \beta_{j+1}/d_j. \end{aligned} \tag{4}$$

The square root is a *complex* square root. In particular the code uses δ_j^2, not $|\delta_j|^2$. With $d_j \neq 0$, the update is

$$w_j = \frac{v_j - u_j w_{j-1} - z_j w_{j-2}}{d_j}, \tag{5}$$

$$x_j = x_{j-1} + c_j \eta_{j-1} w_j, \tag{6}$$

$$\eta_j = -s_j \eta_{j-1}, \qquad \widehat{r}_j = |s_j|\widehat{r}_{j-1} = |\eta_j|. \tag{7}$$

The two stored Lanczos arrays alternate roles. The current array initially holds $\beta_j v_j$ and is normalized to v_j; the old array holds v_{j-1}. Multiplying the old array by β_j therefore produces the last term in Eq. (2). The newly written array is $t_j = \beta_{j+1} v_{j+1}$.

4. Why these updates correspond to elimination

The local transformation is

$$G_j = \begin{bmatrix} c_j & s_j \\ -s_j & c_j \end{bmatrix}, \qquad G_j \begin{bmatrix} \delta_j \\ \beta_{j+1} \end{bmatrix} = \begin{bmatrix} d_j \\ 0 \end{bmatrix}, \qquad G_j^T G_j = I. \tag{8}$$

Embedding these rotations in successive row pairs gives $Q_k \underline{T}_k = \begin{bmatrix} R_k \\ 0 \end{bmatrix}$. R_k has only a diagonal and two superdiagonals: in column j, its nonzero entries are z_j, u_j, d_j. If $W_k = V_k R_k^{-1}$, equating column j in $W_k R_k = V_k$ gives Eq. (5). The transformed RHS has newly fixed entry $c_j \eta_{j-1}$ and remaining entry η_j, yielding Eq. (6).

Important distinction. These rotations are complex-orthogonal ($G_j^T G_j = I$), but generally not unitary ($G_j^\dagger G_j \neq I$). Therefore this elimination is not, in general, a Euclidean minimum-residual solve. The name `pminres_csr` should not be read as a guarantee that $\|b - Ax_j\|_2$ is minimized over the Krylov subspace.

5. Mapping the notation to the source

Mathematics	Fortran	PETSc C port
β_j, β_{j+1}	`beta0, beta1`	`beta, next`
c_{j-1}, c_{j-2}	`gamma0, gamma1` (before update)	same names
s_{j-1}, s_{j-2}	`sigma0, sigma1` (before update)	same names
d_j, u_j, z_j	`rho1, rho2, rho3`	same names
$\eta_j, \widehat{r}_j$	`eta, res`	`eta, estimate`

The rotation variables named `gamma0/gamma1` are *not* the physical broadening γ in Eq. (1).

6. Why the true residual must be checked

After solving the first k transformed equations, the projected residual is $Q_k^T e_{k+1}\eta_k$. Thus, while the Lanczos basis remains orthonormal,

$$\boxed{\|b - Ax_k\|_2 = |\eta_k| \left\|Q_k^T e_{k+1}\right\|_2 .} \tag{9}$$

The code tracks only $\widehat{r}_k = |\eta_k|$. The missing factor need not be one for complex-orthogonal rotations. Loss of Lanczos orthogonality and accumulated roundoff can further separate the estimate from the actual residual. Their disagreement is therefore *not only a floating-point effect*.

A two-dimensional example. Take $H = \mathrm{diag}(-1, 1)$, $\gamma = 0.2$, and $b = (1, 1)^T/\sqrt{2}$. Then $\beta_1 = 1$, $\alpha_1 = 0.2i$, $\beta_2 = 1$, $d_1 = \sqrt{0.96}$, $c_1 = 0.2i/\sqrt{0.96}$, and $s_1 = 1/\sqrt{0.96}$. The implemented first step gives

$$x_1 = \frac{0.2i}{0.96} b, \qquad \widehat{r}_1 = 1.020621, \qquad \|b - Ax_1\|_2 = |s_1|\sqrt{|s_1|^2 + |c_1|^2} = 1.062296. \tag{10}$$

By contrast, minimizing $\|b - tAb\|_2$ over complex t gives $t = -0.2i/1.04$. This illustrates why the implemented shifted recurrence must not be identified with standard Euclidean MINRES.

Actual stopping behavior. Fortran stops when $\widehat{r}_k \le \tau$ (or at its iteration cap), then prints $\|Ax_k - b\|_2$; that final check does not change its always-zero `info`. The PETSc port recomputes the true residual when the estimate first qualifies (and on zero next-Lanczos norm), and accepts convergence only if $\|Ax_k - b\|_2 \le \tau$. Otherwise it continues or reports breakdown/iteration failure. It also checks zero/nonfinite d_j; the production Fortran has no corresponding explicit guard. Complex-orthogonal elimination can break down when $\delta_j^2 + \beta_{j+1}^2 = 0$, even if the two terms are individually nonzero.

7. Same mathematics, different distributed kernels

Fortran's `dot_product(v,q)` computes $v^\dagger q$, followed by an MPI sum. PETSc's convention is `VecDot(x,y)` $= y^\dagger x$, so the equivalent call is `VecDot(work,v,&alpha)`, *not* the reversed argument order. The C update `VecAXPBYPCZ(z,a,b,c,x,y)` means $z \leftarrow ax + by + cz$; it fuses the three-term vector updates above. Both implementations store five work vectors plus the solution and normally use one matrix product and two global scalar reductions per iteration, with extra work for residual checks. GMRES instead stores an expanding basis and orthogonalizes against it.

For $\gamma \neq 0$, a useful accuracy bound follows directly from $A^\dagger A = H^2 + \gamma^2 I$: $\|x_\star - x_k\|_2 \le \|b - Ax_k\|_2 / |\gamma|$. It requires the *true* residual, and becomes less favorable as broadening shrinks. General preconditioning need not preserve the structure used here.

Implementation sources and API references.

Local implementation sources (inspected 5 October 2026):

`src/linsys.f90`; `petsc_shifted_pminres.c`; adapter `petsc_shifted_pminres.py`.

Equations and the worked example are derived directly from these updates. PETSc API conventions: VecDot documentation; VecAXPBYPCZ documentation.